

BookAtlas Final Report

Group 11

Authors & Contributors:

Name	Email
Karine Davtyan	karineda@my.yorku.ca
Angelina De Santis	adsantis@my.yorku.ca
Yawei Dong	ydong77@my.yorku.ca
Manoj Ravichandran	manojr@my.yorku.ca
Samin Sharif	samin23@my.yorku.ca
Owen Zhu	owenz75@my.yorku.ca

Last updated: August 4, 2026

Status: Final

Table Of Contents

Overview.....	3
Requirements Tracking Module (RTM).....	4
Functional.....	4
Non-Functional.....	9
Design Documents.....	12
Overall System Architecture.....	12
Sequence Diagrams.....	14
Class Diagrams.....	19
Demonstration.....	24
Minimum Requirements & Course Concepts.....	24
RTM Requirements Met.....	25
Usability & UI Consistency.....	25
GitHub Repository & SENG Practices.....	25
Testing.....	26
Test Cases & RTM.....	26
Unit Test Cases & Data.....	26
Backend.....	26
Frontend.....	27
Test Case Outcomes.....	27
Backend.....	27
Frontend.....	28
Conclusion.....	29

Overview

BookAtlas is a platform that helps readers organize and engage with their reading. Users can track books they've finished, are currently reading, or want to read, and they can build collections that reflect their interests. Each user also has a public profile where others can view their posts, which helps create a more connected community. Beyond tracking books, users can browse and search for titles, view detailed book information, and participate in discussions. Altogether, the app gives readers a single place to manage their reading and interact with others who share similar interests.

While building BookAtlas, we wanted the project to feel meaningful for users and within the context of the course. For users, our goal was to create a platform that feels organized, intuitive, and genuinely helpful for tracking books and sharing thoughts. For the course, we aimed to apply concepts from lectures wherever they fit naturally, such as in how we modelled data, structured services, or approached analytics and event-driven communication. Even though we didn't use every concept from the lectures, the ones we chose helped us build a cleaner and more thoughtful system. Concepts like service-based architecture and event-sourcing shaped how we approached the project and how we solved problems along the way.

Overall, BookAtlas reflects the goals we set at the start: build something functional, grounded in course concepts, and genuinely useful for readers who want a simple way to track books and participate in discussions about their favourite books.

Website (hosted on Google Cloud Run)

<https://book-atlas-frontend-209112693939.us-west2.run.app>

GitHub Repository

<https://github.com/OwenHZhu/EECS4314-Project>

Video Demo

<https://youtu.be/Il6ulzOEwyl?si=GwynnjCTjCjpWH1n>

Requirements Tracking Module (RTM)

The BookAtlas Requirements Traceability Matrix tracks every testable requirement for the project back to a specific test case. This allows the team to verify system behavior and prove coverage before submission. Rather than one long combined list, the requirements were split across two tabs based on the type of requirement being verified: functional (what the system does) and non-functional (how well it does it). Each tab uses the same nine-column structure:

- Test Case ID
- Requirement ID
- Requirement Description
- Test Type
- Test Data/Input
- Expected Result
- Actual Result
- Status
- Date Tested

Functional

The BookAtlas RTM covers functional requirements describing concrete system behaviors and features. These are things a user can directly do or observe. These are organized by service, using ID prefixes that map to BookAtlas's five backend services:

- AUTH-xx — registration, login/logout, sessions, profile edits
- BOOK-xx — catalog, search, ratings aggregation, external API sync
- LIB-xx — personal library, reading status, wishlist, favourites
- DISC-xx — forums, threads, replies, moderation
- PROF-xx — public profile display and edit permissions

The test cases each track to one functional requirement, with a Unit or Integration test type depending on whether it verifies isolated logic or an end-to-end flow.

Test Case ID	Requirement ID	Requirement Description	Test Type	Test Data / Input	Expected Result	Actual Result	Status	Date Tested
TC-001	AUTH-01	User can register a new account with username, email, and password	Unit	username='reader01', email='reader01@test.com', password='Password0rd!'	Account created; user record stored with hashed password	Matches	Passed	2026-06-30
TC-002	AUTH-02	Frontend validates required fields and password length before	Unit	password='123' (too short); required fields left blank	Inline validation errors shown; request not sent to backend	Matches	Passed	2026-06-30

		submitting registration						
TC-003	AUTH-03	Registration rejects duplicate email or username	Integration	Register with email already in use	409 Conflict; 'email already registered' error returned	Matches	Passed	2026-07-01
TC-004	AUTH-04	User can log in with valid credentials	Integration	email='reader01@test.com', password='Password!'	200 response; JWT/session token issued to frontend	Matches	Passed	2026-07-01
TC-005	AUTH-05	Invalid login credentials show a clear error message	Integration	email='reader01@test.com', password='wrongpass'	401 response; 'invalid email or password' message displayed	Matches	Passed	2026-07-01
TC-006	AUTH-06	Authenticated session/token unlocks protected features after login	Integration	Valid JWT attached to request for /library	Protected route returns 200 and user-specific data	Matches	Passed	2026-07-02
TC-007	AUTH-07	User can log out and token is blacklisted	Integration	POST /logout with active JWT, then reuse same JWT	Logout succeeds; reused JWT is rejected with 401	Matches	Passed	2026-07-02
TC-008	AUTH-08	User can update profile (username, bio, profile picture)	Integration	PATCH /profile with new bio and profile picture file	Profile updated; changes reflected on public profile	Matches	Passed	2026-07-02
TC-009	AUTH-09	User can change their password	Integration	current_password, new_password='NewPass1!'	Password updated; old password no longer authenticates	Matches	Passed	2026-07-02
TC-010	AUTH-10	User can delete their own account	Integration	DELETE /account with confirmation	Account and associated session removed; further logins rejected	Matches	Passed	2026-07-03
TC-011	AUTH-11	Passwords are hashed before storage, never stored in plaintext	Unit	Inspect users table after registration	Password column contains bcrypt/argon2 hash, not plaintext	Matches	Passed	2026-07-03
TC-012	BOOK-01	User can search for books by title	Integration	query='Dune'	Returns book cards matching title 'Dune'	Matches	Passed	2026-07-04
TC-013	BOOK-02	User can search for books by author	Integration	query='Brandon Sanderson'	Returns book cards by matching author	Matches	Passed	2026-07-05

TC-014	BOOK-03	User can search for books by genre	Integration	query='Fantasy'	Returns book cards tagged with the Fantasy genre	Matches	Passed	2026-07-05
TC-015	BOOK-04	Only the specified number of search results are returned	Unit	limit=50	Returns 50 or less records	Matches	Passed	2026-07-06
TC-016	BOOK-05	Search results display cover, title, author, genre, and average rating	Unit	Render search result list for query='Dune'	Each book card shows cover image, title, author, genre, avg rating	Matches	Passed	2026-07-06
TC-017	BOOK-06	Selecting a book opens its details page	Integration	Click on book card for 'Dune'	Navigates to book details page with description, rating, status options	Matches	Passed	2026-07-07
TC-018	BOOK-07	Book catalog stores title, author, genre, cover image, description, ISBN	Unit	Create book record via admin/catalog endpoint	Book stored with all required metadata fields populated	Matches	Passed	2026-07-07
TC-019	BOOK-08	Global average rating is computed via aggregate query, not stored separately	Integration	3 users rate a book 4, 5, 3 stars	Average rating returned as 4.0, computed live from ratings table	Matches	Passed	2026-07-08
TC-020	BOOK-09	Books can be added, updated, or removed from the global catalog	Integration	POST/PUT/DELETE on /books/{id}	Catalog reflects create/update/delete operation correctly	Matches	Passed	2026-07-08
TC-021	BOOK-10	External API integration fetches book covers and metadata	Integration	Add new book by ISBN via Google Books/OpenLibrary lookup	Cover image and metadata auto-populated from external API	Matches	Passed	2026-07-09
TC-022	LIB-01	Authenticated user can add a book to their personal library	Integration	user_id=42, book_id=7, POST /library	Library entry created linking user and book	Matches	Passed	2026-07-05
TC-023	LIB-02	User can assign a reading status (Currently Reading, Completed, Dropped)	Unit	status='Currently Reading'	Library entry status field updated and persisted	Matches	Passed	2026-07-05
TC-024	LIB-03	User can update the reading status of a library entry	Integration	Change status from 'Currently Reading' to 'Completed'	Status updated; timestamp of change recorded	Matches	Passed	2026-07-06

TC-025	LIB-04	User can remove a book from their personal library	Integration	DELETE /library/{book_id}	Library entry removed; book no longer appears in user's library	Matches	Passed	2026-07-06
TC-026	LIB-05	Library page can be filtered by reading status, author, timestamp, and title	Unit	Apply filters: status='Dropped'	Only matching library entries are displayed	Matches	Passed	2026-07-06
TC-027	LIB-06	A user cannot have duplicate library records for the same book	Integration	Add same book_id twice for same user_id	Second request rejected with 409 or upsert applied, no duplicate row	Matches	Passed	2026-07-07
TC-028	LIB-07	User can only rate and favourite dropped or finished books	Integration	POST with "wishlist" status	No rating or is_favourite is applied	Matches	Passed	2026-07-07
TC-029	LIB-08	User can add a book to Favourites independently of reading status	Integration	POST with is_favourite = true	Book appears in Favourites list	Matches	Passed	2026-07-08
TC-030	LIB-09	A book can belong to multiple lists simultaneously (e.g., Favourites AND Currently Reading)	Integration	Mark book as Favourite while status = Currently Reading	Book appears in both Favourites and Currently Reading views	Matches	Passed	2026-07-08
TC-031	LIB-10	User can rate a book from 1 to 5 stars, with only one active rating per book	Unit	rating=5 submitted twice for same book	Second submission updates the existing rating rather than creating a new one	Matches	Passed	2026-07-08
TC-032	LIB-11	User can update or remove their own rating	Integration	PATCH rating to 3, then DELETE rating	Rating updates to 3; after delete, rating no longer counted in average	Matches	Passed	2026-07-09
TC-033	LIB-12	start_date and end_date track reading progress; end_date cannot precede start_date	Unit	start_date='2026-01-10', end_date='2026-01-05'	Validation error returned; end_date must be >= start_date	Matches	Passed	2026-07-09
TC-034	DISC-01	Discussion forums are	Integration	GET /forums/thread	Returns threads scoped only to	Matches	Passed	2026-07-12

		organized by individual book		s?book_id={book_id}	the specified book			
TC-035	DISC-02	Discussion forums are organized by tag	Integration	POST /forums/threads with tags=[array of tags]	Each thread has tags associated with it	Matches	Passed	2026-07-13
TC-036	DISC-03	User can create a thread for a given book	Integration	book_id={book_id}	Thread created and book field populated	Matches	Passed	2026-07-14
TC-037	DISC-04	Thread stores title, content, author, tags, related book, has_spoilers, and timestamps	Unit	Create thread with all fields populated	All thread fields persisted and retrievable	Matches	Passed	2026-07-15
TC-038	DISC-05	Authenticated users can reply to threads	Integration	POST /forums/threads/{thread_id}/replies with valid JWT	Reply created and associated with the thread	Matches	Passed	2026-07-16
TC-039	DISC-06	Unauthenticated users may view threads but cannot reply	Integration	POST /forums/threads/{thread_id}/replies with no auth token	401 Unauthorized returned; thread remains viewable via GET	Matches	Passed	2026-07-17
TC-040	DISC-07	Replies support a nested/hierarchical structure via optional parent reply	Unit	Create reply with parent_reply_id set	Reply nested correctly under its parent in the thread tree	Matches	Passed	2026-07-17
TC-041	DISC-08	Users can only edit or delete threads/replies they wrote	Integration	user_id=5 attempts to delete thread authored by user_id=9	403 Forbidden returned; deletion blocked for non-author	Matches	Passed	2026-07-18
TC-042	DISC-09	Spoiler-flagged content is hidden by default until the user chooses to reveal it	Unit	Thread/post flagged spoiler=true	Content rendered blurred/hidden until user clicks reveal	Matches	Passed	2026-07-19
TC-043	DISC-10	Threads can be searched by book	Integration	GET /forums/threads?s?book_id={book_id}	Returns threads for the given book	Matches	Passed	2026-07-20
TC-044	DISC-11	User IDs for posts/likes are resolved server-side from the verified token, not the request body	Unit	Submit like/reply with a spoofed user_id in request body	Server ignores body user_id; action attributed to authenticated token owner	Matches	Passed	2026-07-21

TC-046	PROF-01	Public profile displays username and posts	Integration	GET /profile/{username}	Public profile page renders all specified fields	Matches	Passed	2026-07-18
TC-047	PROF-02	Private information (email, credentials) never appears on public profiles	Unit	Inspect public profile API response payload	Response contains no email or authentication/credential fields	Matches	Passed	2026-07-20
TC-048	PROF-03	Users can edit only their own profile	Integration	user_id=5 attempts PATCH /profile/{other_user_id}	403 Forbidden returned; only profile owner can edit	Matches	Passed	2026-07-23

Non-Functional

The RTM for Non-Functional objectives are more about the underlying reliability, security, and performance characteristics that shape the user's experience. Rather than checking whether a specific action works correctly, these test cases verify how well the system behaves under real-world conditions: whether it stays secure, responds quickly, recovers gracefully from failure, and scales without breaking. This is tracked using a single NFR-xx prefix (NFR-01 through NFR-14). These include:

- HTTPS enforcement
- Graceful handling of database outages
- Response-time thresholds under load
- Independent service deployability
- Fault isolation between services
- Caching of expensive queries

Test Case ID	Requirement ID	Requirement Description	Test Type	Test Data / Input	Expected Result	Actual Result	Status	Date Tested
TC-049	NFR-01	Passwords are stored as a secure hash, never in plaintext or reversible form	Unit	Inspect stored password value after registration	Stored value is a bcrypt/argon2 hash, not readable plaintext	Matches	Passed	2026-07-09
TC-050	NFR-02	Requests over plain HTTP are redirected to HTTPS	Integration	Request http://api.bookatlas.app/health	Request redirected (301/308) to https:// equivalent	Matches	Passed	2026-07-10
TC-051	NFR-03	Each service exposes a	Integration	GET /health on auth, book,	200 response with status 'ok'	Matches	Passed	2026-07-11

		health-check endpoint reporting system status	n	library, discussion services	for each healthy service			
TC-052	NFR-04	System returns a graceful error instead of crashing on DB connection failure	Integration	Simulate Supabase/DB connection outage, then call any endpoint	500-level error with clear message; service does not crash or hang	Matches	Passed	2026-07-12
TC-053	NFR-05	Average API response time stays within an acceptable threshold under normal load	Performance	Run load test with 50 concurrent GET /library or /books requests	Average response time remains under defined threshold (e.g., 500ms)	Matches	Passed	2026-07-14
TC-054	NFR-06	Services can be deployed and scaled independently of one another	Integration	Redeploy Book Service only, without redeploying Auth/Library/Discussion	Book Service updates successfully; other services remain unaffected	Matches	Passed	2026-07-15
TC-055	NFR-07	System continues functioning if one service fails (fault isolation)	Integration	Take Discussion Service offline while Library Service is running	Users can still access and manage their personal library	Matches	Passed	2026-07-16
TC-056	NFR-08	CORS is configured correctly to support secure frontend-backend integration	Integration	Request from frontend origin vs. an untrusted origin	Allowed origin succeeds; untrusted origin is blocked by CORS policy	Matches	Passed	2026-07-17
TC-057	NFR-09	Event-driven actions (e.g., SearchExecuted, BookViewed) are processed asynchronously without delaying API responses	Integration	Trigger a book search while monitoring API response time	Event published to RabbitMQ; API response time unaffected by event processing	Matches	Passed	2026-07-18
TC-058	NFR-10	Profile picture storage/retrieval via the data lake (Databricks) is secure and reliable	Integration	Upload, retrieve, and delete a profile picture by user ID	Image stored/retrieved/deleted correctly; unauthorized access blocked	Matches	Passed	2026-07-19
TC-059	NFR-11	Switching between List View and Grid View on the	Unit	Toggle view switch button on library page with 100+	View switches smoothly with no visible delay or layout break	Matches	Passed	2026-07-21

		library page renders without noticeable lag		books loaded				
TC-060	NFR-12	Automated unit/integration tests exist and pass for each backend service	Integration	Run pytest suite for auth, book, library, and discussion services	All test suites execute and pass with no failing cases	Matches	Passed	2026-07-22
TC-061	NFR-13	Heavy aggregation queries (e.g., analytics) are cached to reduce repeated load	Performance	Call analytics endpoint twice in quick succession	Second call served from cache (cachetools) with lower latency	Matches	Passed	2026-07-23
TC-062	NFR-14	Search queries return results quickly enough to feel instant to the user	Performance	Submit a book search query and measure time to first result	Results returned within a short, user-perceived 'instant' threshold	Matches	Passed	2026-07-24

Design Documents

Overall System Architecture

BookAtlas is a dynamic web-application built with a React frontend and a set of backend services that handle the core domains of the application. The frontend manages page navigation, form validation, and user interactions, and communicates with the backend through Axios API clients. Whenever the frontend sends a request, it validates the inputs first, and then the backend service validates the request again, performs the necessary operations, and queries the database before returning a response. After authentication, the backend issues a JWT, which the frontend uses to access protected features such as personal libraries, profiles, and discussion forums.

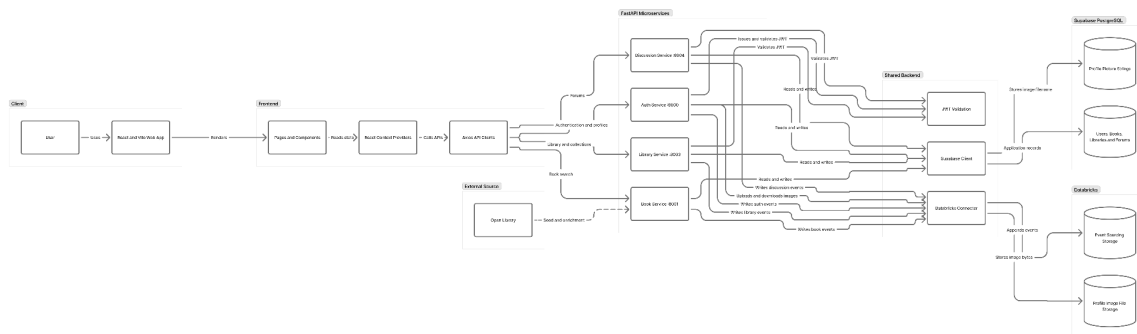
The backend follows a service-based architecture, where each domain, such as authentication, books, libraries, and discussions, is implemented as its own service. These services run independently but share common middleware for JWT validation, database access, and event publishing. They communicate through shared identifiers (`user_id`, `book_id`, etc.), which keeps the system cohesive without introducing the overhead of a full microservices architecture. This approach gives us improved modularity and separation of concerns while still keeping the system simple to test and deploy.

All core transactional data is stored in a monolithic PostgreSQL database hosted on Supabase. This includes users, books, libraries, ratings, threads, and other core entities. Keeping a single relational database helps maintain consistency across services and simplifies schema management.

In addition to the main database, the system includes an analytics service backed by Databricks. Each backend service publishes events to this service, which stores event history in a data lake. This event-sourcing approach allows the system to distinguish between transactional and analytical data, support asynchronous communication, and enable richer analytics features in the future. Concepts from reactive applications and enterprise integration shaped this part of the architecture.

Overall, the architecture balances modularity, maintainability, costs, and scalability. Each service remains focused on its own domain, the shared database keeps the system consistent, and the analytics service provides a foundation for event-driven features without complicating the core application.

Low-Level Architecture Diagram



Note: You can view a clearer architecture diagram in our [GitHub repository](#) under [/coursework/diagrams](#)

This diagram shows how requests flow from the user, through the frontend, and into the backend services, which share database connections, JWT validation, and event-publishing logic.

When a user interacts with BookAtlas, the request begins in the React frontend. The frontend handles form and input validation first, and then sends the request through one of its Axios API clients. These clients route the request to the appropriate backend service based on the feature the user is accessing. For example, the Auth service for login, the Library service for library entries, the Book service for book searches, or the Discussion service for posts and comments.

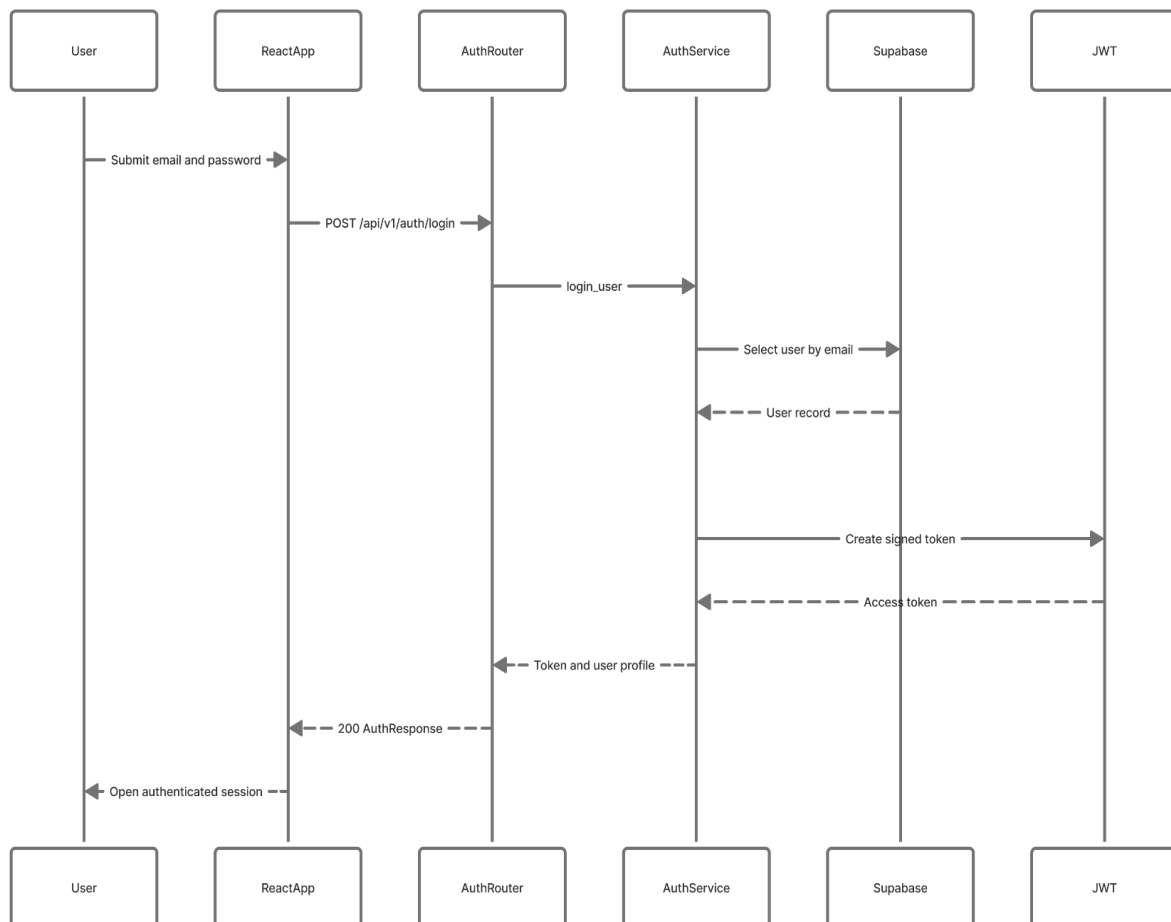
Once the request reaches a backend service, that service performs its own validation, processes the business logic, and interacts with Supabase through the shared database client. If the request involves authentication, the shared JWT middleware verifies the user's token before any protected operations are performed. If the request triggers an event (such as creating a post, updating a library, or viewing a book), the service also publishes an event to the analytics service, which stores it in Databricks.

After the service finishes its work (eg. reading data, updating records, or enriching information) it returns a structured response back to the frontend. The frontend then updates its React state or context providers and re-renders the page or component so the user immediately sees the result of their action.

Sequence Diagrams

For each service, we modelled the core user interaction rather than every possible request. This keeps the diagrams focused and avoids repetitive flows, while still illustrating the full lifecycle of a request and its corresponding response.

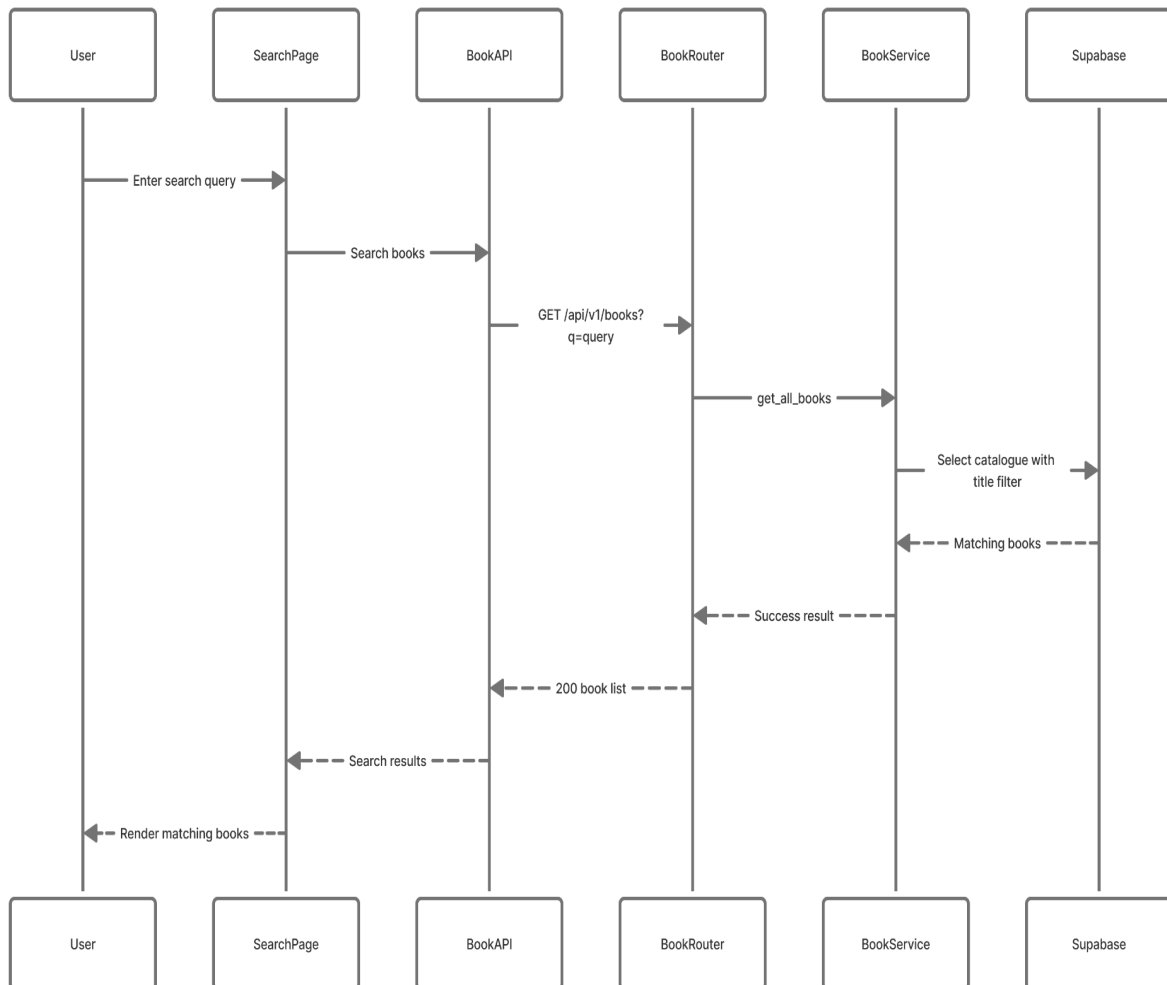
Auth Service - Login



This sequence diagram illustrates how an existing user logs into the BookAtlas system. The user submits an email and password through the frontend, which sends a POST `/api/v1/auth/login` request to the Auth Service. The router validates the request and passes the credentials to the authentication service.

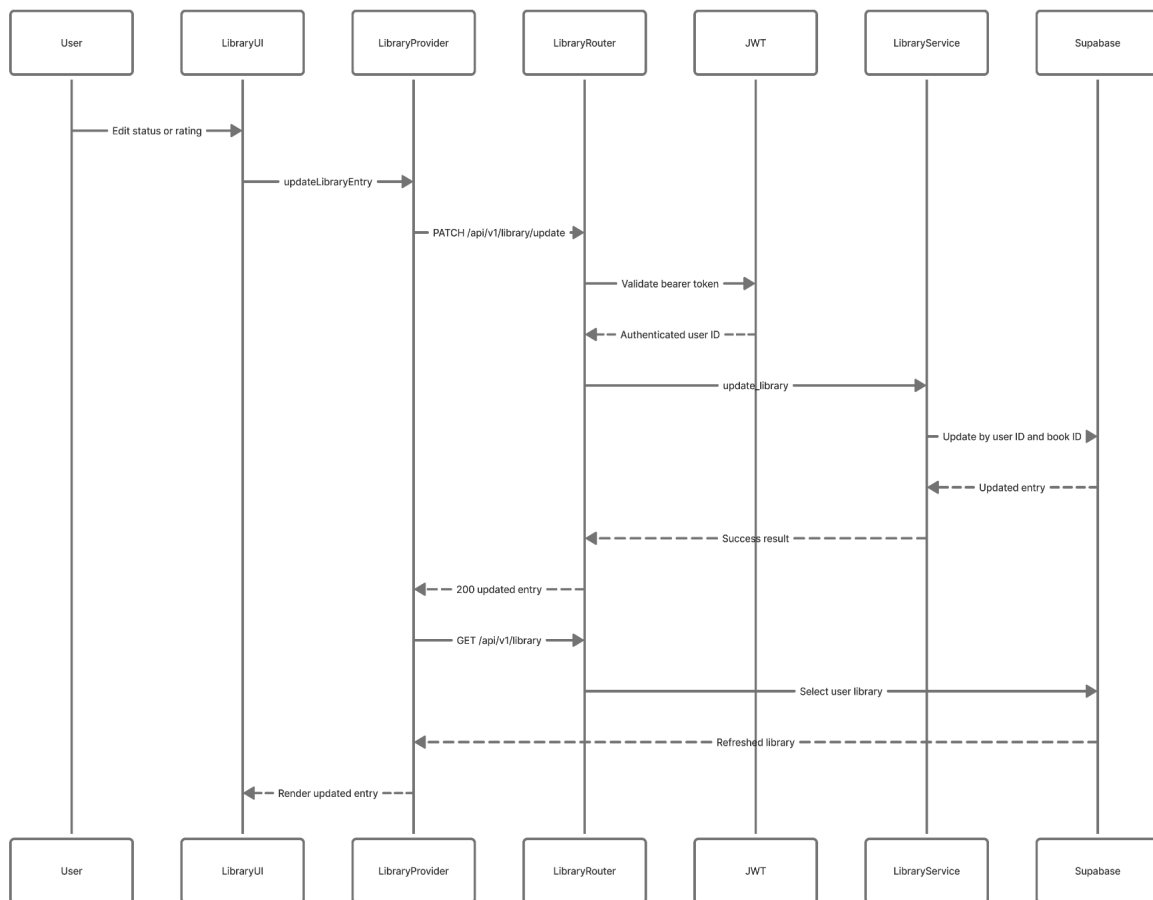
The service layer retrieves the matching user record from Supabase and verifies the submitted password against the stored password hash. After successful verification, the Auth Service generates a JWT and returns it with the user's profile to the frontend. The frontend stores the authentication information and updates the interface to show that the user is logged in.

Book Service - Get Book



This sequence diagram illustrates how a user retrieves the details of a specific book. The frontend sends a request containing the book ID to the Book Service. The router validates the request and passes the ID to the service layer, which queries Supabase for the matching book record.

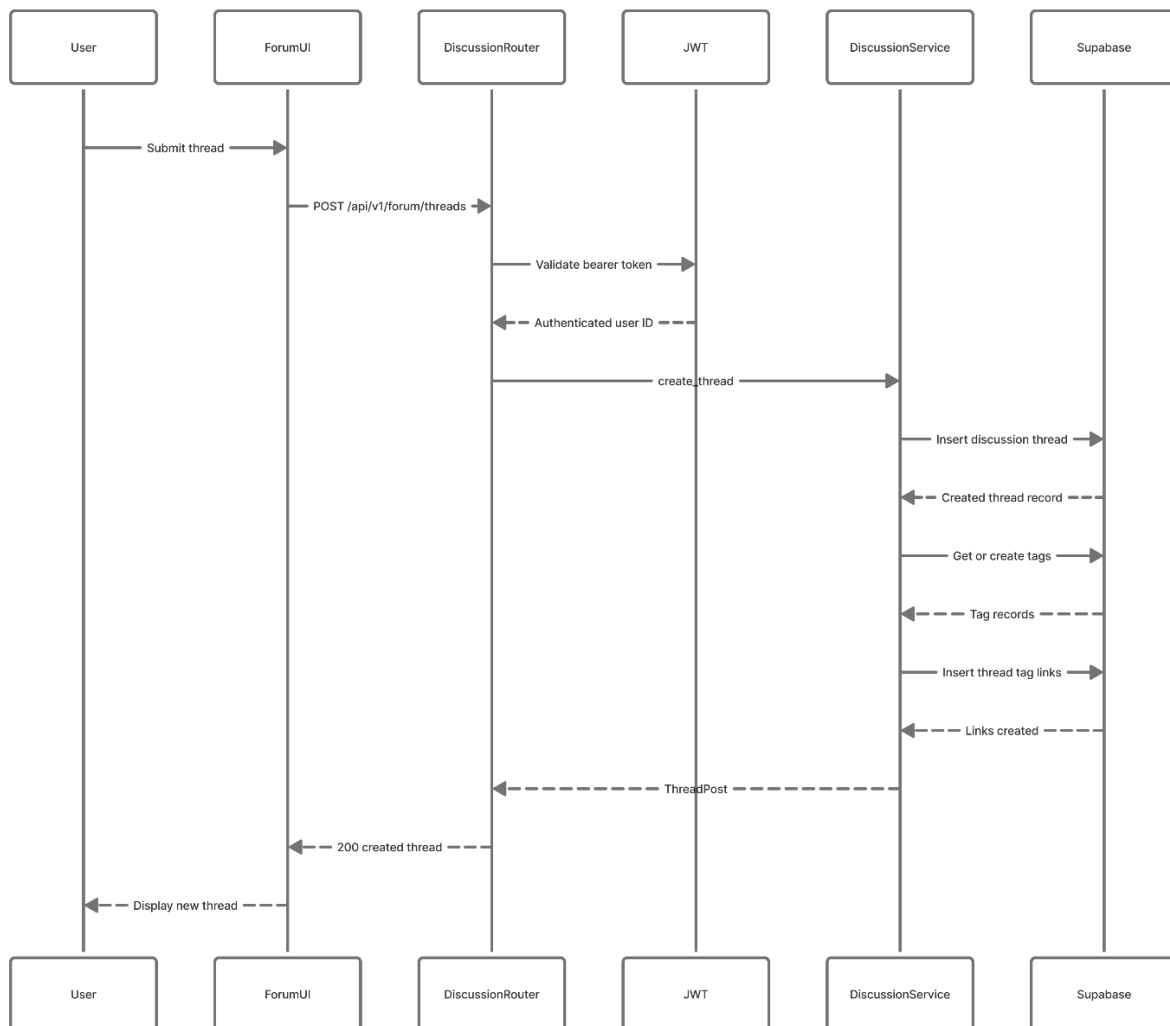
Supabase returns the book data to the service layer, and the Book Service sends the response back to the frontend. The frontend then displays information such as the book's title, author, description, genres, rating, and cover image to the user.

Library Service - Update Book

This diagram illustrates the sequence of interactions that occurs when an authenticated user updates a book in their personal Library. An update may include changing the reading status, rating, or favourite status of the book. The interaction begins when the user submits a change through the Library user interface. The frontend calls `updateLibraryEntry()` in `LibraryProvider`, which sends a `PATCH` request to the Library Service. The request includes the user's JWT in the Authorization header but does not include a user ID supplied by the client.

The Library Router validates the JWT and extracts the authenticated user ID. It then passes the validated request data and user ID to the Library Service layer. The service updates the corresponding database record using both `user_id` and `book_id`, ensuring that a user cannot modify another user's Library entry.

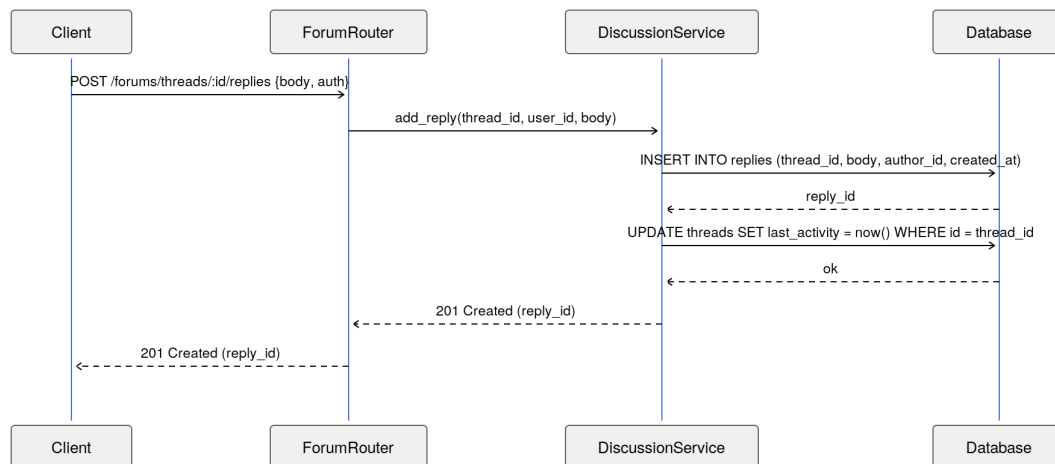
After Supabase returns the updated record, the response is propagated through the service and router layers to the frontend. The Library Provider then retrieves the latest Library data and updates the shared React state, causing the List or Grid view to render the updated entry.

Discussion Service - Post Thread

This sequence diagram illustrates how an authenticated user creates a new discussion thread. The user submits the thread content through the frontend, which sends a request containing the thread information and authentication token to the Discussion Service. The router validates the request and uses the token to identify the user.

The service layer creates the thread record in Supabase and associates it with the authenticated user. Supabase returns the newly created thread data to the Discussion Service, which sends it back to the frontend. The frontend then updates the discussion page to display the new thread.

Discussion Service - Reply to Thread



This sequence diagram shows the flow for adding a reply to a discussion thread in BookAtlas. The Client sends a POST request to `/forums/threads/:id/replies` with the reply body and auth token, which the ForumRouter passes to the DiscussionService as an `add_reply` call with the thread ID, user ID, and body. The DiscussionService inserts the new reply into the Database and receives back a `reply_id`, then updates the parent thread's `last_activity` timestamp so the thread reflects the most recent activity. Once the database confirms the update, the DiscussionService returns the `reply_id` up through the ForumRouter, and the Client receives a 201 Created response confirming the reply was successfully added.

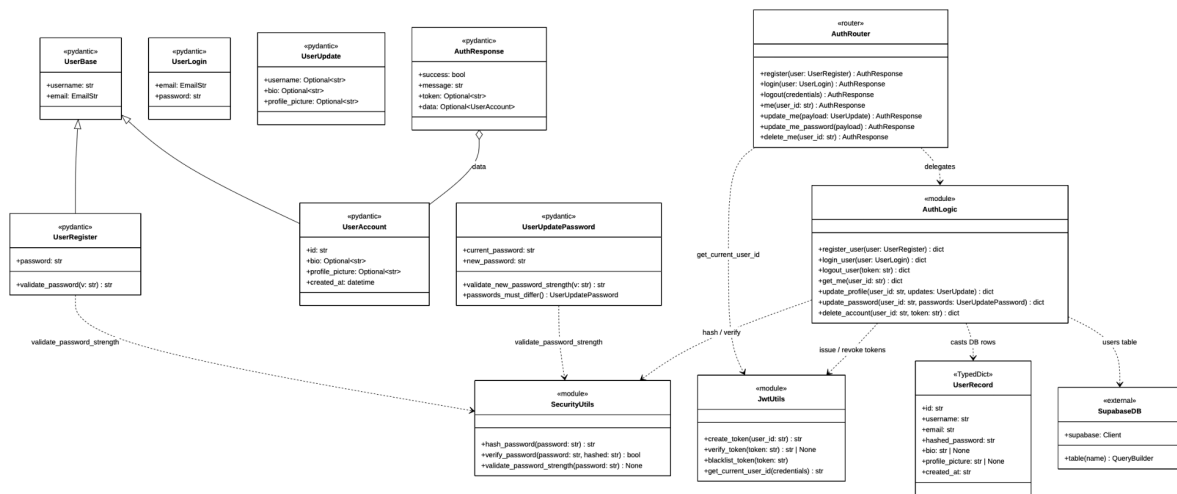
Class Diagrams

These class diagrams demonstrate how each service's components are structured and interact with one another.

Common notations across the diagram are:

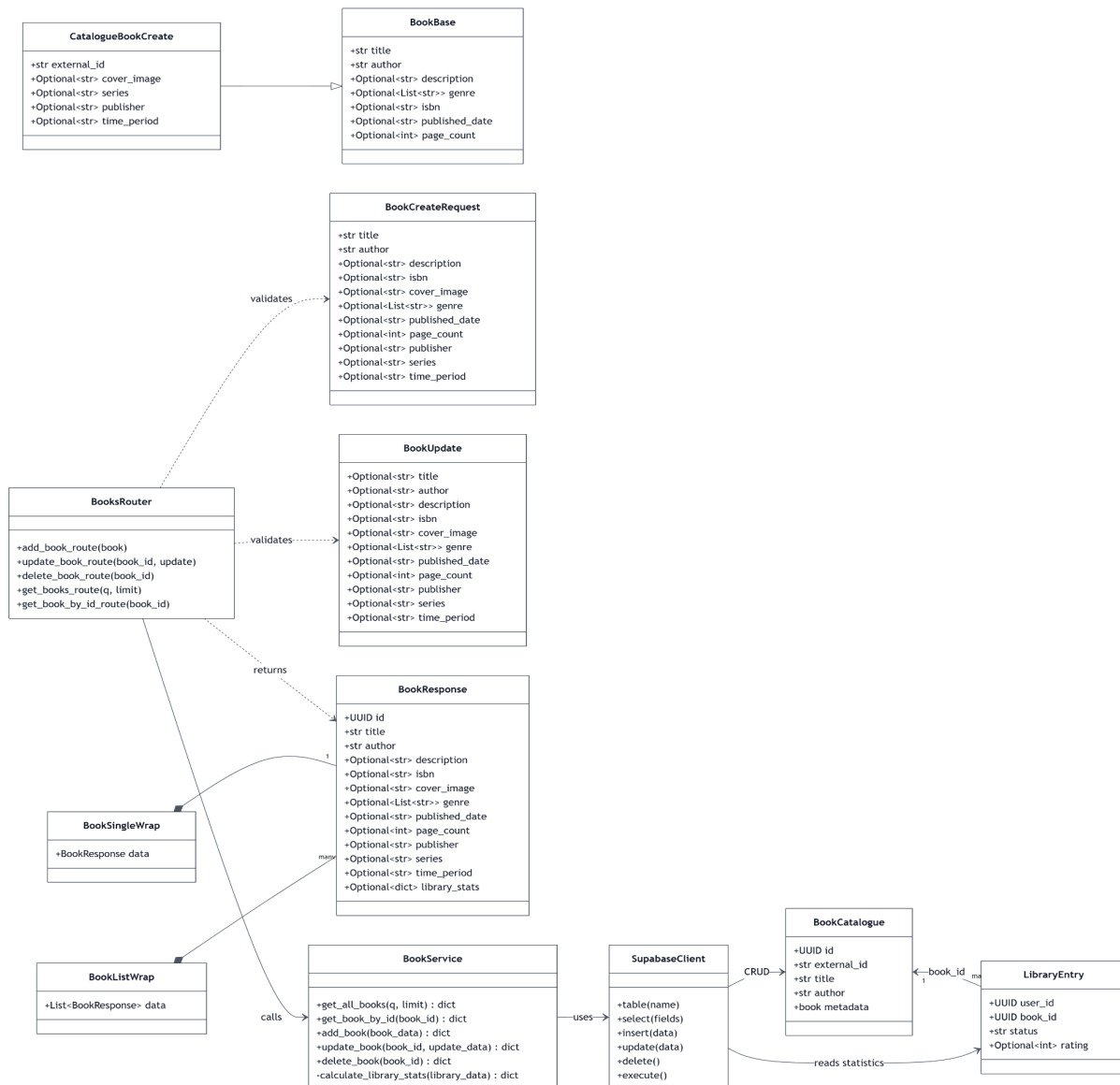
- <<pydantic>> indicates models/schemas used for request and response validation
- <<router>> indicates the HTTP layer that handles requests from the frontend
- <<modules>> indicates service or private helper files that handle database interactions, validation, and business logic
- <<external>> software components outside of the current service, such as the databases

Auth Service



This Auth Service diagram shows the backend/auth_service module of BookAtlas. This service is mainly responsible for user registration, login, JWT-based sessions, and profile management. The AuthRouter exposes HTTP endpoints (register, login, logout, me, update_me, delete_me) and delegates business logic to AuthLogic which coordinates password hashing/verification and validation through Security utils. Additionally, request and response data are validated using pydantic models and enforce specific rules like password strength, user validation and more.

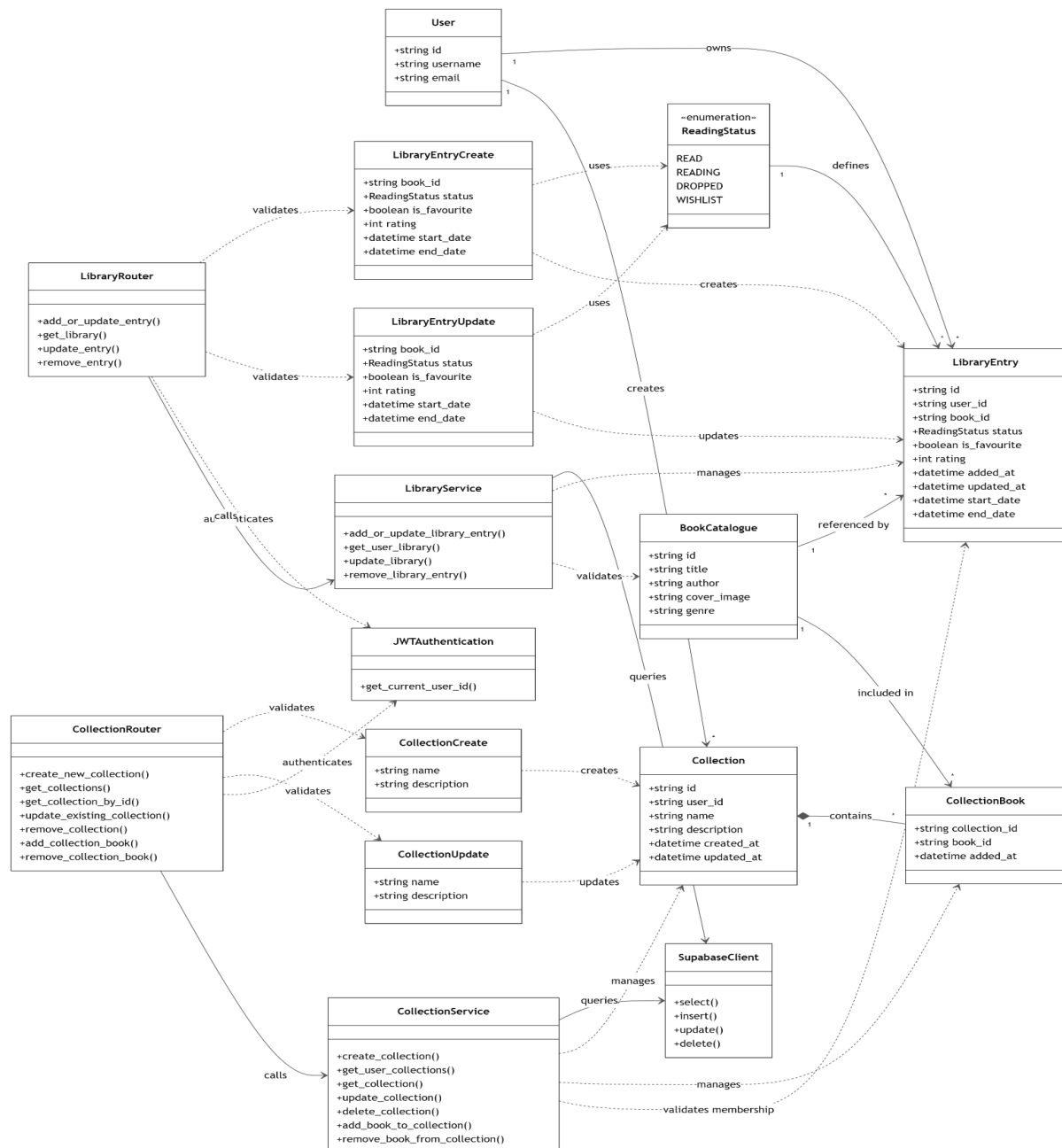
Book Service



The Book Service class diagram presents the structure of the schemas, router, service layer, and database entities used to manage the global book catalogue. BookBase contains the shared book attributes, while the create, update, and response schemas define the data accepted and returned by the API. The wrapper classes represent responses containing either one book or a list of books.

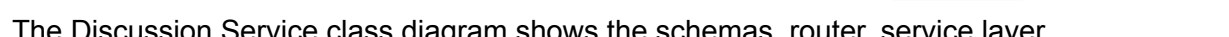
The BooksRouter validates HTTP requests and calls the appropriate functions in BookService. The service layer contains the CRUD logic and uses the Supabase client to access the BookCatalogue table. When retrieving a specific book, it also reads rating and reading-status information from LibraryEntry and calculates aggregated library statistics. This layered design separates validation, request handling, business logic, and database access, improving maintainability and testability.

Library Service

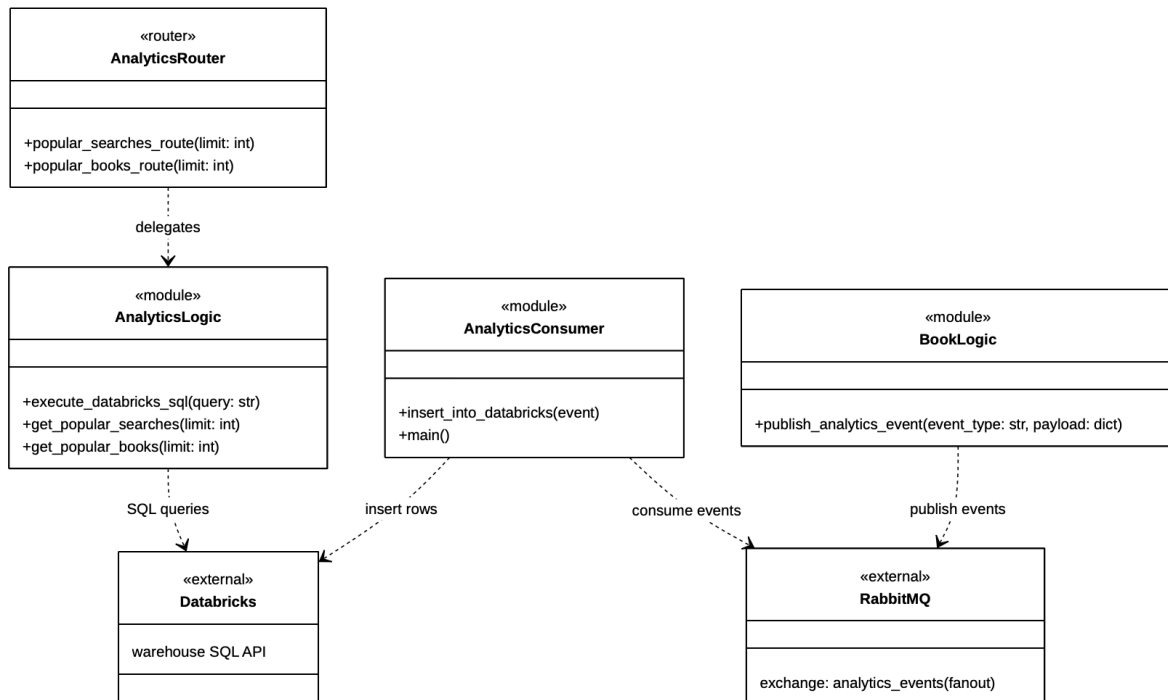


This diagram presents the class structure of the Library Service. A User can own multiple LibraryEntry objects, while each LibraryEntry references one BookCatalogue record and uses one ReadingStatus value. The diagram also shows the Collection many-to-many relationship, where CollectionBook acts as the junction entity between Collection and BookCatalogue.

The backend follows a layered design. Request schemas validate incoming data, routers handle HTTP requests and JWT authentication, and service classes implement business logic and communicate with Supabase.



Analytics Service



This class diagram covers the Analytics Service, which tracks trends like book searches, profile views, authentication patterns, forum activity, and library activity. It is our datalake event-sourcing module where we track key user actions for each interaction on our website for better monitoring and recommendations. The **AnalyticsRouter** exposes two endpoints and delegates to **AnalyticsLogic**, which runs SQL queries against **Databricks** to compute results. Data gets into **Databricks** asynchronously: **BookLogic** publishes analytics events to a **RabbitMQ** exchange, and a separate **AnalyticsConsumer** module picks up those events and inserts them into **Databricks**. This keeps analytics decoupled from the main request flow, so tracking never slows down the user-facing API.

Demonstration

Minimum Requirements & Course Concepts

Throughout this course, we paid close attention to the lectures and constantly discussed how to apply the material to our project. Each time, we considered whether a concept was realistic to implement given our scope and timeframe. While we didn't use everything, the course concepts we included meaningfully improved the project.

From the lectures on monolithic vs distributed architecture, we chose a service-based architecture. This let us keep a monolithic database, improve testability and deployability, and give teammates clear domains to work within.

We also kept the data modelling lecture in mind when designing our entities and relationships. Since Supabase uses PostgreSQL, we followed relational best practices, such as using junction tables for many-to-many relationships and normalizing tables with foreign keys. This kept our database clean and predictable.

From the SDLC lectures, we followed an Agile delivery cycle, focusing on small integrations and revising services when needed rather than pushing fully polished features all at once. This helped with testing and debugging, since the frontend wasn't blocked while the backend added features, and the frontend could easily request additional data or functionality. We also followed the SDLC structure of designing, implementing, and then testing each service.

Our Cyber Security lecture emphasized password security, so our authentication service enforced strict password requirements to reflect that concept.

For data conversion, we mostly used the one-to-one pattern. However, some endpoints, such as the Library and Book services, enriched the target structure with additional details (book info or rating info) before returning it to the frontend. This gave the frontend everything it needed to render pages without extra calculations or API calls.

Most significantly, our analytics service incorporates concepts from data warehouses, data lakes, reactive applications, and enterprise application integration. We separated transactional and analytical data by using Supabase as our OLTP database and Databricks as our OLAP database, since data lakes are ideal for large amounts of structured or unstructured data. Databricks stores event history, helping us shift from synchronous communication to asynchronous, event-driven communication, which is common in reactive systems for improving responsiveness. This is ultimately what moves our app beyond a traditional CRUD project and toward something more enterprise-level.

Overall, the lecture material shaped our project in meaningful ways. Even when we couldn't implement everything, the concepts we chose helped us build a stronger, more thoughtful system that reflects the core ideas of the course.

RTM Requirements Met

The BookAtlas RTM covers 62 test cases (48 functional and 14 non-functional). All test cases were executed and passed, with actual results matching expected results across every requirement (AUTH, BOOK, LIB, DISC, and PROF services, as well as all 14 NFRs). Testing was completed and logged between when the project first started until the end date. There were no failed, blocked or outstanding cases that needed attention. As a result, the requirements defined for the lifespan of the BookAtlas project were fully met.

Usability & UI Consistency

Although BookAtlas is similar to GoodReads, we wanted it to feel noticeably different in terms of UI. We leaned toward a dark-academia style, using muted or neutral reds, browns, and grays, with small pops of colour for visual interest. This made sense to us because atlases, as referenced in the name, are tied to history and dark-academia themes, and most people use dark-mode on their devices anyway.

We also leaned toward a more modular layout, where each component sits in its own rounded box. This helped keep the interface visually organized and made it easier to distinguish one component from another.

Usability was another priority. We added a global navigation bar, used modals for more complex actions, and made sure button labels were clear. While designing the pages in Figma, we also labelled user-interaction flows so we could picture how people would move through the site. All of these choices worked together to make the website easy to navigate and to ensure users always understood what they were doing when they interacted with something.

GitHub Repository & SENG Practices

We organized BookAtlas into multiple independent services, including a separate frontend, backend, analytics, and shared module directories. This allowed team members to work on different parts of the system without affecting each other.

Additionally, we followed consistent folder structures for all services which included routes, models, and API endpoints. We also documented the project through a detailed README, architecture diagrams, setup instructions, and API documentation. This made it easier to onboard teammates to other services and run the application locally.

Our GitHub workflow used feature branches and pull requests to keep development organized. Each feature or bug fix was created in its own branch and thoroughly reviewed via checking out the feature branch before being merged into the main branch. This helped reduce merge conflicts and keep the codebase stable.

Testing

Test Cases & RTM

The RTM tracks test cases by mapping each one directly to a specific requirement using unique IDs (AUTH, BOOK, LIB, DISC, PROF) for functional requirements, and a single NFR prefix for non-functional ones. Every test case records the requirement it traces to, whether it's a Unit or Integration test, and other metadata such as the input data, expected vs. actual result, and pass/fail status. Each test was directly followed and tested. The edge cases were tracked and reported at each stage. This structure lets us confirm both that every requirement has a test and that every test serves a documented purpose.

Unit Test Cases & Data

Backend

Each backend service was tested using pytest, FastAPI's TestClient, and mocked Supabase responses. Mocking isolates each service from the real database, which keeps tests repeatable and prevents test data from ever touching production data.

Across all services, tests are organized into three main layers:

- Schema Tests: Validate request and response models, including required fields, optional fields, field-length restrictions, invalid values, and default timestamps.
- Service Tests: Verify business logic and database operations, such as creating, retrieving, updating, and deleting records.
- Router Tests: Send simulated HTTP requests to API endpoints and verify response status codes, returned data, request validation, and error handling.

Each service's full test suite is run independently with pytest, giving us clear pass/fail output for every requirement. These results are then checked against the RTM to confirm that every functional and non-functional requirement has a corresponding test, and that the actual pytest outcome matches the expected result documented in the matrix. This keeps the RTM as a living source of truth, if a requirement changes or a new one is added, we know exactly which pytest cases need to be written or updated to keep coverage accurate.

Frontend

Frontend testing was completed using Vitest, React Testing Library, and jsdom. The tests used sample user, book, and library data to check how the interface behaved.

The frontend tests were a mix of unit, component, and integration tests. Smaller functions and hooks were tested separately, while full pages were tested with their related components and mocked dependencies.

The tests for frontend covered the main parts of the system:

- Authentication: We tested the login and registration pages, form validation, error messages, and redirects after successful authentication.
- Books and Discover Page: We tested book cards, searching, genre filtering, opening the book details modal, and actions such as rating or adding a book to the library.
- Book Page: We tested loading and error states, book information, community ratings, library statistics, favourites, reading status, and user ratings.
- Library: We tested the library page, tabs, list and grid views, sorting, filtering, editing entries, deleting entries, and library state management.

We also tested supporting hooks, context providers, validation functions, and API helper functions. Backend service calls were mocked so the frontend could be tested independently without needing every backend service to be running.

Test Case Outcomes

These are screenshots demonstrating the outcomes of our test cases.

Backend

Auth Service Test Outcomes

```
auth_service/tests/test_security.py::TestPasswordStrengthValidation::test_missing_number_raises PASSED
auth_service/tests/test_security.py::TestPasswordStrengthValidation::test_missing_special_char_raises PASSED
auth_service/tests/test_security.py::TestPasswordStrengthValidation::test_all_rules_broken_reports_all_of_them PASSED
auth_service/tests/test_security.py::TestPasswordStrengthValidation::test_empty_password_raises PASSED
auth_service/tests/test_update_password.py::TestUpdatePassword::test_correct_current_password_succeeds PASSED
auth_service/tests/test_update_password.py::TestUpdatePassword::test_incorrect_current_password_is_rejected PASSED
auth_service/tests/test_update_password.py::TestUpdatePassword::test_nonexistent_user_returns_not_found PASSED
auth_service/tests/test_update_password.py::TestUpdatePassword::test_update_write_failure_is_reported PASSED
===== 50 passed in 58.80s =====
```

Book Service Test Outcomes

```
book_service/tests/test_schemas_book.py::test_book_create_valid PASSED
book_service/tests/test_schemas_book.py::test_book_create_invalid_title PASSED
book_service/tests/test_services_book.py::test_add_book_success PASSED
book_service/tests/test_services_book.py::test_update_book_drops_none_values PASSED
book_service/tests/test_services_book.py::test_calculate_library_stats_empty PASSED
book_service/tests/test_services_book.py::test_calculate_library_stats_with_data PASSED
===== 8 passed in 5.32s =====
```

Library Service Test Outcomes

```
library_service/tests/test_services_library.py::test_update_library_no_data PASSED [ 94%]
library_service/tests/test_services_library.py::test_update_library_not_found PASSED [ 96%]
library_service/tests/test_services_library.py::test_remove_library_entry_success PASSED [ 98%]
library_service/tests/test_services_library.py::test_remove_library_entry_not_found PASSED [100%]
===== 58 passed, 1 warning in 0.84s =====
```

Discussion Service Test Outcomes

```
collected 4 items

discussion_service/tests/test_db_inserts.py::test_create_thread_inserts PASSED [ 25%]
discussion_service/tests/test_db_inserts.py::test_create_reply_inserts PASSED [ 50%]
discussion_service/tests/test_likes.py::test_like_thread_and_count PASSED [ 75%]
discussion_service/tests/test_likes.py::test_toggle_thread_like_uses_unlike_when_already_liked PASSED [100%]
===== 4 passed in 0.59s =====
```

Frontend

The full frontend suite completed successfully:

```
Test Files 19 passed (19)
Tests      214 passed (214)
```

All authentication, book, discover, library, hook, provider, service-helper, and validation tests passed.

Conclusion

Overall, we're happy with our project and the knowledge we gained from it. Throughout the process, we pushed ourselves to integrate course concepts we were initially unfamiliar with, such as the requirements tracking module (RTM), service-based architecture, and event-sourcing. Engaging with these ideas strengthened our system-design and problem-solving skills. Even though we already understood product backlogs from Agile/Scrum, the RTM added complexity by requiring us to track test cases and features in far greater detail. Similarly, adopting service-based architecture and implementing event-sourcing required substantial refactoring and even a small test project before we could finalize our design.

We also handled several difficult technical decisions as a team. Switching from MongoDB to Supabase, restructuring our services to properly follow service-based architecture, and integrating Databricks and event-sourcing late in the project all required careful discussion and coordination. These choices were challenging, but they ultimately strengthened the project.

Beyond the technical work, we learned how to collaborate effectively. Our weekly meetings mirrored an Agile/Scrum environment and helped us stay accountable for our tasks. Outside of those meetings, we held additional calls to work together and stayed connected through Discord. We also managed blockers well. For example, if the frontend was waiting on a backend service, they used that time to test, refactor, or scaffold other pages. Meanwhile, the backend team consistently explored additional features once their core work was complete, ensuring no one was ever idle.

Looking ahead, if we were to continue developing the project, we would prioritize a notification system and a followers/following feature. Allowing users to be notified when others interact with their posts or comments would create a more complete discussion experience. Likewise, enabling users to follow each other would help build a more engaged community and encourage ongoing participation. Even so, we're proud of the core features we delivered within the short timeframe of this project.